UNIVERSITI TEKNOLOGI MARA

Cawangan Perak
Kampus Tapah

FACULTY OF SCIENCES COMPUTER AND MATHEMATICS

DIPLOMA IN SCIENCES COMPUTER (CSC110)

FUNDAMENTALS OF DATA STRUCTURES (CSC248)

FINAL REPORT

SDG THEME : INDUSTRY, INNOVATION AND INFRASTURCTURE

FACTORY INVENTORY MANAGEMENT SYSTEM

PREPARED BY

NO.	NAME	STUDENT ID
1.	SAIFUL ALLIF NAADHZIR BIN SAIFUL HAZMI	2024216492
2.	MUHAMMAD ALI WAFAR BIN BEHERAN	2024218726
3.	MUHAMMAD FARIS AQIL BIN MOHAMAD ARIFF	2024213742
4.	MUHAMAD ANAS AZHARI BIN MARKHANI	2024293072

PREPARED FOR:

DR. MOHD. FAAIZIE BIN DARMAWAN

SUBMISSION DATE:

23 JANUARY 2026

TABLE OF CONTENT

1.0 INTRODUCTION PROJECT

In many factories, managing inventory efficiently is a constant challenge. Companies often struggle with inaccurate stock records, poor tracking of products, and delays in updating inventory information. Without a centralized system, employees may rely on manual records or scattered files, leading to errors, wasted resources, and disruptions in production. As a result, even when resources are available, the lack of proper coordination and data management reduces overall industrial efficiency.

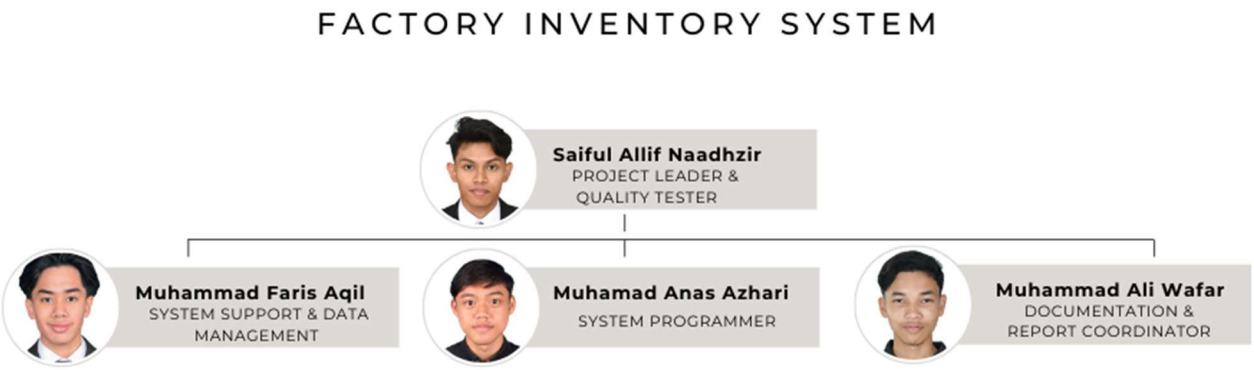
To address this issue, the proposed Java based Factory Inventory Management System aims to provide a centralized platform for managing factory products and stock information. The system allows users to record product details, monitor quantities, update prices, and categorize items in real time. By organizing inventory data in a structured and reliable way, the system improves accuracy, supports faster decision-making, and enhances day-to-day factory operations. This initiative supports SDG 9: Industry, Innovation and Infrastructure by promoting the use of digital solutions to strengthen industrial processes and infrastructure.

The system is built around key components such as products, categories, and inventory records. It uses object-oriented programming and data structure concepts to store and manage product information efficiently. By bringing all inventory data into one system, the application makes inventory management simpler, more transparent, and more dependable, helping factories operate more effectively and sustainably.

Objectives :

1. To provide a centralized platform for managing factory inventory information.
2. To improve the accuracy and reliability of inventory records.
3. To assist users in performing routine inventory tasks more efficiently.
4. To support better monitoring and control of factory resources.
5. To organize factory product and inventory information in a single system.

1.1 GROUP MEMBERS AND TASK DISTRIBUTION



1.2 PROJECT ROLES AND RESPONSIBILITIES

1. **PROJECT LEADER & SYSTEM QUALITY TESTER**

   Responsible for leading the project team and overseeing the overall development of the system. Ensures that project objectives are met, coordinates tasks among team members, and performs system testing to verify functionality, accuracy, and reliability before final submission.

2. **SYSTEM PROGRAMMER**

   Responsible for developing the core system functionalities, including implementing system logic, handling inventory operations, and ensuring that the system runs correctly according to the project requirements.

3. **DOCUMENTATION & REPORT COORDINATOR**

   Responsible for preparing project documentation, including the proposal, report writing, and organizing project content. Ensures that all written materials are clear, well-structured, and aligned with the system objectives.

4. **SYSTEM SUPPORT & DATA MANAGEMENT**

   Responsible for managing system data, preparing test data files, and assisting in system testing. Provides support during development by helping identify errors and ensuring data consistency within the system.

2.0 COMPLETE CODING OF ALL CLASSES

2.1 FactoryInventorySystem (main class)

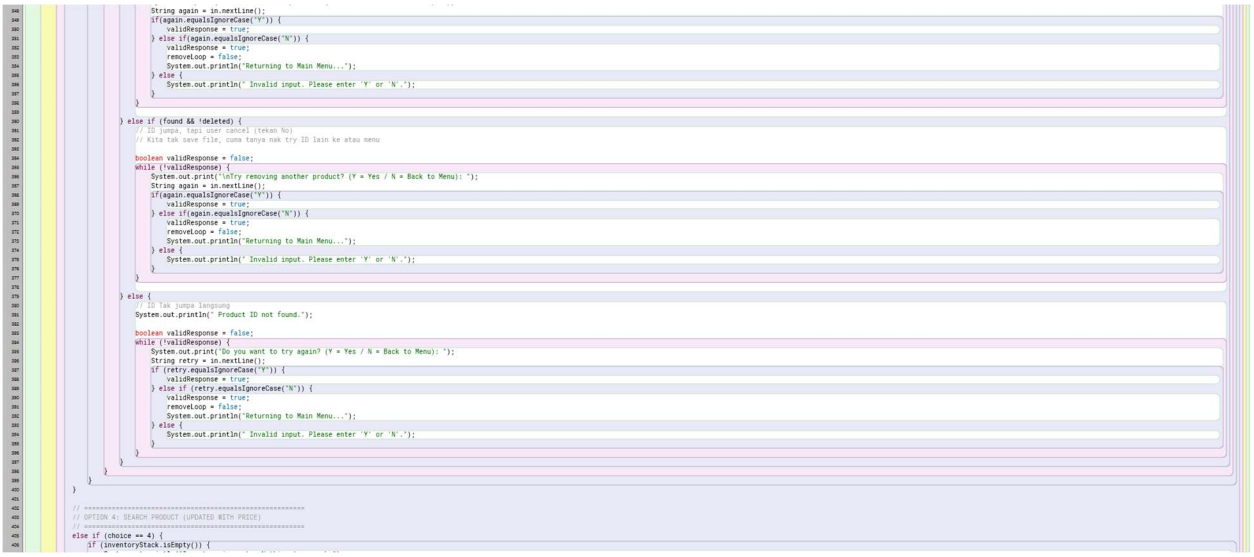
```java
import java.util.Scanner;
import java.util.StringTokenizer;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;


public class FactoryInventorySystem {

    static String fileName = "Homelander.txt";

    // ============================================================
    // METHOD TO SAVE FILE
    // ============================================================
    public static void saveFile(Stack<Product> productStack) {
        try {
            FileWriter fw = new FileWriter(fileName, false);

            Stack<Product> tempStack = new Stack<>();

            // 1. Inventory -> Temp
            while (!productStack.isEmpty()) {
                tempStack.push(productStack.pop());
            }

            // 2. Temp -> Inventory + Write File
            while (!tempStack.isEmpty()) {
                Product p = tempStack.pop();

                fw.write(
                    p.getProductID() + ";" +
                    p.getProductName() + ";" +
                    p.getProductQuantity() + ";" +
                    p.getPrice() + ";" +
                    p.getCategory() + "\n"
                );

                productStack.push(p);
            }

            fw.close();
        } catch (IOException e) {
            System.out.println("Oops, error while saving file!");
        }
    }

    public static void main(String[] args) {
        Stack<Product> inventoryStack = new Stack<>();
        Stack<Product> recentActivityStack = new Stack<>();
        Stack<Product> tempStack = new Stack<>();

        Scanner in = new Scanner(System.in);

        // ============================================================
        // 1. LOAD DATA FROM FILE
        // ============================================================
        try {
            File f = new File(fileName);
            if (!f.exists()) {
                f.createNewFile();
            }

            Scanner fileScanner = new Scanner(f);
            while (fileScanner.hasNextLine()) {
                String line = fileScanner.nextLine();
                StringTokenizer st = new StringTokenizer(line,";");

                String id = st.nextToken();
                String name = st.nextToken();
                int qty = Integer.parseInt(st.nextToken());
                double price = Double.parseDouble(st.nextToken());
                String cat = st.nextToken();

                Product p = new Product(id, name, qty, price, cat);

                inventoryStack.push(p);
            }
            fileScanner.close();
        } catch (Exception e) {
            System.out.println("Error reading file, continuing...");
        }

        int choice = -1;

        while (choice != 0) {
            System.out.println("\n========================================");
            System.out.println("          Factory Inventory System ");
            System.out.println("========================================");
            System.out.println("1. Display Inventory");
            System.out.println("2. Add New Product");
            System.out.println("3. Remove Product ");
            System.out.println("4. Search Product ");
            System.out.println("5. Update Product");
            System.out.println("6. View Recent Activity");
            System.out.println("0. Exit");
            System.out.print("Your choice: ");

            try {
                choice = Integer.parseInt(in.nextLine());
            } catch (Exception e) {
                choice = -1;
            }

            // ============================================================
            // OPTION 1: DISPLAY
            // ============================================================
            if (choice == 1) {
                if (inventoryStack.isEmpty()) {
                    System.out.println("Inventory is empty.");
                } else {
                    System.out.println("\n--- PRODUCT LIST (STACK) ---");
                    System.out.println("-----------------------------------------------------------------------------------------");
                    System.out.printf("| %-10s | %-25s | %-8s | %-12s | %-20s |%n",
                            "ID", "Product Name", "Qty", "Price", "Category");
                    System.out.println("-----------------------------------------------------------------------------------------");

                    while (!inventoryStack.isEmpty()) {
                        Product p = inventoryStack.pop();
```

```
            while (!inventoryStack.isEmpty()) {
                Product p = inventoryStack.pop();
                System.out.printf("| %-10s | %-25s | %-8d | RM %-9.2f | %-20s |%n",
                                  p.getProductID(),
                                  p.getProductName(),
                                  p.getProductQuantity(),
                                  p.getPrice(),
                                  p.getCategory());
                tempStack.push(p);
            }
            System.out.println("-------------------------------------------------------------------------------");

            while (!tempStack.isEmpty()) {
                inventoryStack.push(tempStack.pop());
            }
        }
        System.out.println("\n[Press Enter to return to Main Menu...]");
        in.nextLine();
    }

// ========================================================
    // OPTION 2: ADD NEW PRODUCT
    // ========================================================
    else if (choice == 2) {
        boolean addLoop = true;

        while (addLoop) {

            System.out.print("\nEnter Product ID: ");
            String id = in.nextLine();

            boolean idExists = false;

            while(!inventoryStack.isEmpty()){
                Product p = inventoryStack.pop();
                if(p.getProductID().equalsIgnoreCase(id)){
                    idExists = true;
                }
                tempStack.push(p);
            }
            while(!tempStack.isEmpty()){
                inventoryStack.push(tempStack.pop());
            }

            if (idExists) {
                System.out.println("This ID already exists.");

                boolean validResponse = false;
                while (!validResponse) {
                    System.out.print("Try another ID? (Y/N): ");
                    String retry = in.nextLine();
                    if (retry.equalsIgnoreCase("Y")) validResponse = true;
                    else if (retry.equalsIgnoreCase("N")) {
                        validResponse = true;
                        addLoop = false;
                        System.out.println("Returning...");
                    }
                }
```

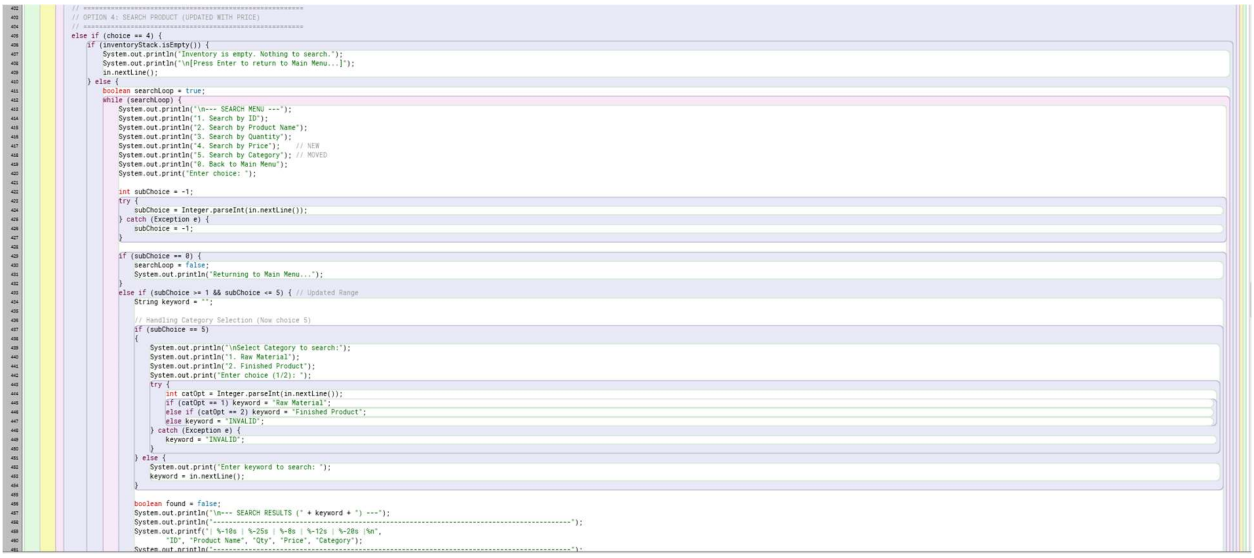
```
            }
        } else {
            // --- STEP 2: NAME VALIDATION (INI YANG BARU) ---
            // Kita masuk sini sebab ID dah LULUS (unik).
            // Sekarang kita nak check Nama pula.

            String name = "";
            boolean nameUnique = false; // Flag untuk loop nama

            while (!nameUnique) {
                System.out.print("Product Name: ");
                name = in.nextLine();

                boolean nameExists = false;

                // 1. Tuang Keluar & Check Nama
                while (!inventoryStack.isEmpty()) {
                    Product p = inventoryStack.pop();

                    // Check kalau nama sama (Case insensitive: 'Simen' == 'simen')
                    if (p.getProductName().equalsIgnoreCase(name)) {
                        nameExists = true;
                    }
                    tempStack.push(p);
                }

                // 2. Tuang Balik (Wajib!)
                while (!tempStack.isEmpty()) {
                    inventoryStack.push(tempStack.pop());
                }

                // 3. Keputusan
                if (nameExists) {
                    System.out.println("Error: Product Name '" + name + "' already exists! Please use a different name.");
                } else {
                    nameUnique = true; // Nama unik, boleh keluar dari loop ni
                }
            }

            // --- STEP 3: MINTA DATA LAIN (SAMA MACAM BIASA) ---
            System.out.print("Quantity: ");
            int qty = Integer.parseInt(in.nextLine());
            System.out.print("Price: ");
            double price = Double.parseDouble(in.nextLine());

            // Category Validation
            String cat = "";
            boolean validCat = false;
            while (!validCat) {
                System.out.println("Category: 1.Raw Material, 2.Finished Product");
                System.out.println("Choose (1/2): ");
                try {
                    int catOpt = Integer.parseInt(in.nextLine());
                    if (catOpt == 1) { cat = "Raw Material"; validCat = true; }
                    else if (catOpt == 2) { cat = "Finished Product"; validCat = true; }
                    else System.out.println("Invalid choice.");
                } catch (Exception e) {
                    System.out.println("Invalid number.");
                }
```

```
                    System.out.println("Invalid number.");
                }
            }

            // --- STEP 4: SAVE ---
            Product newP = new Product(id, name, qty, price, cat);

            inventoryStack.push(newP);
            recentActivityStack.push(newP);
            saveFile(inventoryStack);
            System.out.println("Product added successfully!");

            // Tanya user nak tambah barang lagi ke tak...
            boolean validResponse = false;
            while (!validResponse) {
                System.out.print("\nAdd another product? (Y/N): ");
                String again = in.nextLine();
                if (again.equalsIgnoreCase("Y")) validResponse = true;
                else if (again.equalsIgnoreCase("N")) {
                    validResponse = true;
                    addLoop = false;
                    System.out.println("Returning to Main Menu...");
                }
            }
        }
    }

    // ==========================================================
    // OPTION 3: REMOVE PRODUCT (WITH CONFIRMATION)
    // ==========================================================
    else if (choice == 3) {
        if (inventoryStack.isEmpty()) {
            System.out.println("Inventory is empty. Nothing to remove.");
            System.out.println("\n[Press Enter to return to Main Menu...]");
            in.nextLine();
        } else {
            boolean removeLoop = true;
            while (removeLoop) {

                // --- DISPLAY LIST (Sama macam kod asal) ---
                System.out.println("\n--- LIST OF PRODUCTS AVAILABLE TO REMOVE ---");
                System.out.println("-----------------------------------------------------------------------------------------");
                System.out.printf("| %-10s | %-25s | %-8s | %-12s | %-20s |%n",
                        "ID", "Product Name", "Qty", "Price", "Category");
                System.out.println("-----------------------------------------------------------------------------------------");

                while (!inventoryStack.isEmpty()) {
                    Product p = inventoryStack.pop();
                    System.out.printf("| %-10s | %-25s | %-8d | RM %-9.2f | %-20s |%n",
                                      p.getProductID(),
                                      p.getProductName(),
                                      p.getProductQuantity(),
                                      p.getPrice(),
                                      p.getCategory());
                    tempStack.push(p);
                }
                System.out.println("-----------------------------------------------------------------------------------------");

                while (!tempStack.isEmpty()) {
```

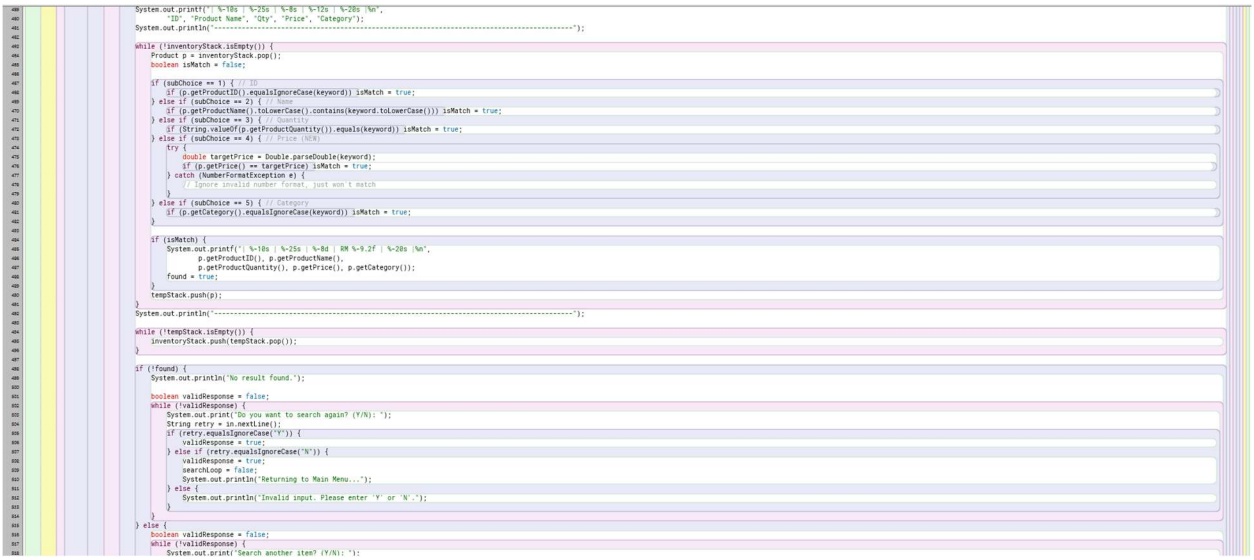
```
                while (!tempStack.isEmpty()) {
                    inventoryStack.push(tempStack.pop());
                }
                // ------------------------------------------------

                System.out.print("\nEnter Product ID to remove: ");
                String removeId = in.nextLine();

                boolean found = false;
                boolean deleted = false; // Flag baru: untuk tahu user jadi delete ke tak
                Product removedProduct = null;

                while (!inventoryStack.isEmpty()) {
                    Product p = inventoryStack.pop();

                    // Check ID
                    if (p.getProductID().equalsIgnoreCase(removeId)) {
                        found = true; // ID wujud

                        // --- BAHAGIAN BARU: CONFIRMATION ---
                        System.out.println("\nProduct Found: " + p.getProductName());
                        System.out.print("Are you sure you want to delete? (Y/N): ");
                        String confirm = in.nextLine();

                        if (confirm.equalsIgnoreCase("Y")) {
                            // User confirm NAK delete
                            deleted = true;
                            removedProduct = p;
                            recentActivityStack.push(p);
                            System.out.println("Product removed from inventory.");
                            // KITA TAK PUSH KE TEMPSTACK (Biar hilang)
                        } else {
                            // User CANCEL (tekan N atau lain-lain)
                            System.out.println("Deletion cancelled. Product kept safe.");
                            tempStack.push(p); // Simpan balik product ni
                        }

                    } else {
                        tempStack.push(p); // Product lain selamatkan macam biasa
                    }
                }

                // Kemas balik barang yang terselamat
                while (!tempStack.isEmpty()) {
                    inventoryStack.push(tempStack.pop());
                }

                // --- LOGIK LEPAS SEARCH ---
                if (deleted) {
                    // Kalau betul-betul delete, baru save file
                    saveFile(inventoryStack);
                    System.out.println("Successfully updated database: " + removedProduct.getProductName() + " is gone.");

                    // Tanya nak remove lagi tak (Sama flow macam kod asal)
                    boolean validResponse = false;
                    while (!validResponse) {
                        System.out.print("\nRemove another product? (Y = Yes / N = Back to Menu): ");
                        String again = in.nextLine();
                        if(again.equalsIgnoreCase("Y")) {
```



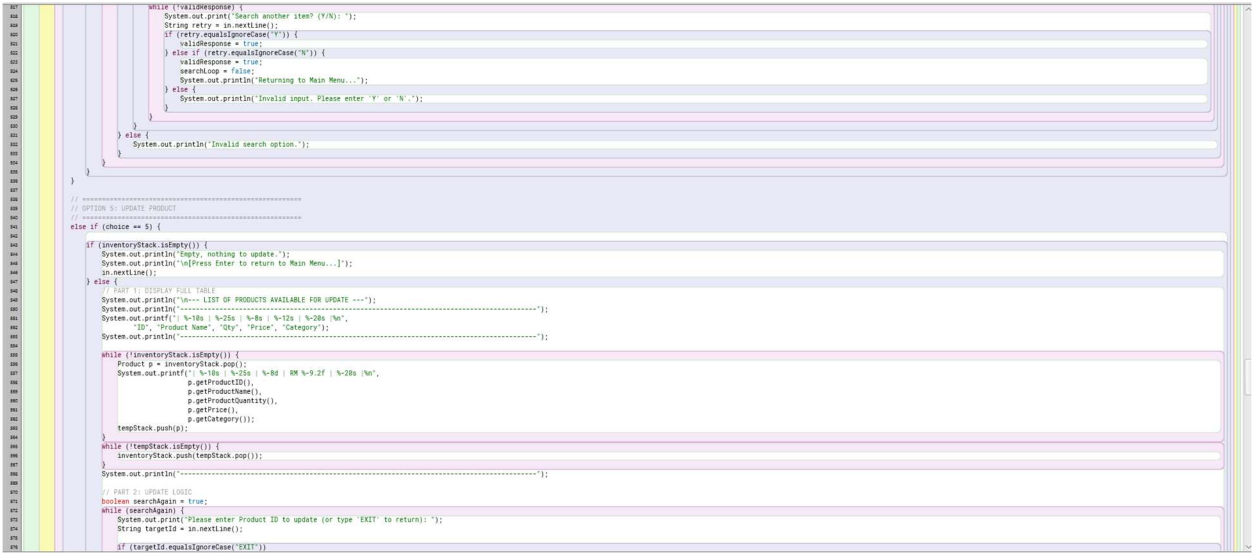
```
                String again = in.nextLine();
                if(again.equalsIgnoreCase("Y")) {
                    validResponse = true;
                } else if(again.equalsIgnoreCase("N")) {
                    validResponse = true;
                    removeLoop = false;
                    System.out.println("Returning to Main Menu...");
                } else {
                    System.out.println(" Invalid input. Please enter 'Y' or 'N'.");
                }
            }

        } else if (found && !deleted) {
            // ID jumpa, tapi user cancel (tekan No)
            // Kita tak save file, cuma tanya nak try ID lain ke atau menu

            boolean validResponse = false;
            while (!validResponse) {
                System.out.print("\nTry removing another product? (Y = Yes / N = Back to Menu): ");
                String again = in.nextLine();
                if(again.equalsIgnoreCase("Y")) {
                    validResponse = true;
                } else if(again.equalsIgnoreCase("N")) {
                    validResponse = true;
                    removeLoop = false;
                    System.out.println("Returning to Main Menu...");
                } else {
                    System.out.println(" Invalid input. Please enter 'Y' or 'N'.");
                }
            }

        } else {
            // ID Tak jumpa langsung
            System.out.println(" Product ID not found.");

            boolean validResponse = false;
            while (!validResponse) {
                System.out.print("Do you want to try again? (Y = Yes / N = Back to Menu): ");
                String retry = in.nextLine();
                if (retry.equalsIgnoreCase("Y")) {
                    validResponse = true;
                } else if (retry.equalsIgnoreCase("N")) {
                    validResponse = true;
                    removeLoop = false;
                    System.out.println("Returning to Main Menu...");
                } else {
                    System.out.println(" Invalid input. Please enter 'Y' or 'N'.");
                }
            }
        }
    }
}

// ============================================================
// OPTION 4: SEARCH PRODUCT (UPDATED WITH PRICE)
// ============================================================
else if (choice == 4) {
    if (inventoryStack.isEmpty()) {
```



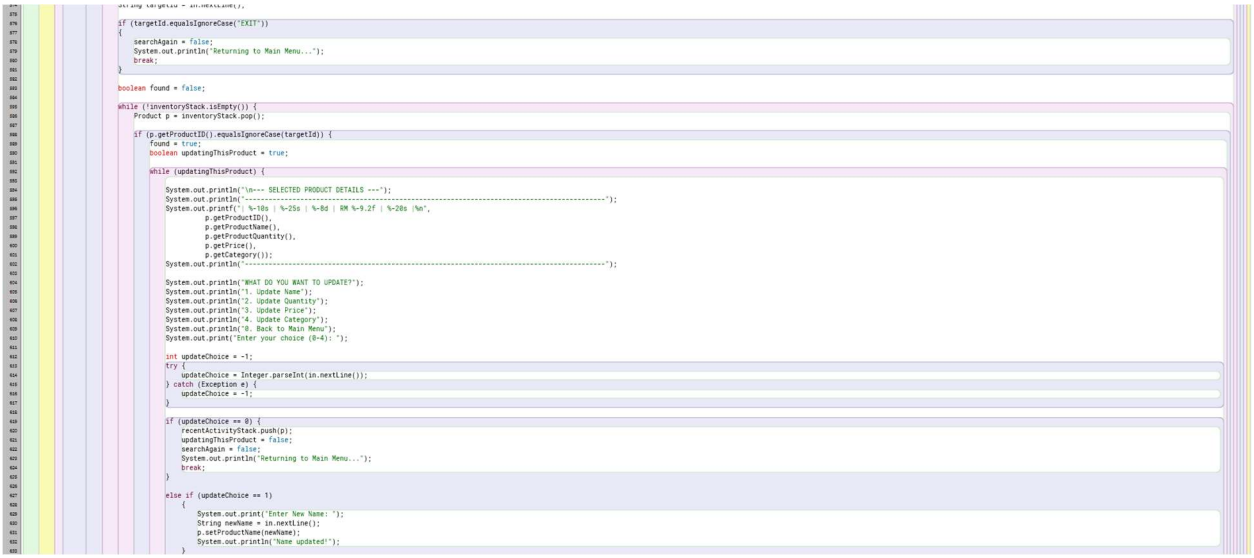
```
// ============================================================
// OPTION 4: SEARCH PRODUCT (UPDATED WITH PRICE)
// ============================================================
else if (choice == 4) {
    if (inventoryStack.isEmpty()) {
        System.out.println("Inventory is empty. Nothing to search.");
        System.out.println("\n[Press Enter to return to Main Menu...]");
        in.nextLine();
    } else {
        boolean searchLoop = true;
        while (searchLoop) {
            System.out.println("\n--- SEARCH MENU ---");
            System.out.println("1. Search by ID");
            System.out.println("2. Search by Product Name");
            System.out.println("3. Search by Quantity");
            System.out.println("4. Search by Price");   // NEW
            System.out.println("5. Search by Category"); // MOVED
            System.out.println("0. Back to Main Menu");
            System.out.print("Enter choice: ");

            int subChoice = -1;
            try {
                subChoice = Integer.parseInt(in.nextLine());
            } catch (Exception e) {
                subChoice = -1;
            }

            if (subChoice == 0) {
                searchLoop = false;
                System.out.println("Returning to Main Menu...");
            }
            else if (subChoice >= 1 && subChoice <= 5) { // Updated Range
                String keyword = "";

                // Handling Category Selection (Now choice 5)
                if (subChoice == 5)
                {
                    System.out.println("\nSelect Category to search:");
                    System.out.println("1. Raw Material");
                    System.out.print("2. Finished Product");
                    System.out.print("Enter choice (1/2): ");
                    try {
                        int catOpt = Integer.parseInt(in.nextLine());
                        if (catOpt == 1) keyword = "Raw Material";
                        else if (catOpt == 2) keyword = "Finished Product";
                        else keyword = "INVALID";
                    } catch (Exception e) {
                        keyword = "INVALID";
                    }
                } else {
                    System.out.print("Enter keyword to search: ");
                    keyword = in.nextLine();
                }

                boolean found = false;
                System.out.println("\n--- SEARCH RESULTS (" + keyword + ") ---");
                System.out.println("-----------------------------------------------------------------------------------------");
                System.out.printf("| %-10s | %-25s | %-8s | %-12s | %-20s |%n",
                        "ID", "Product Name", "Qty", "Price", "Category");
                System.out.println("-----------------------------------------------------------------------------------------");
```



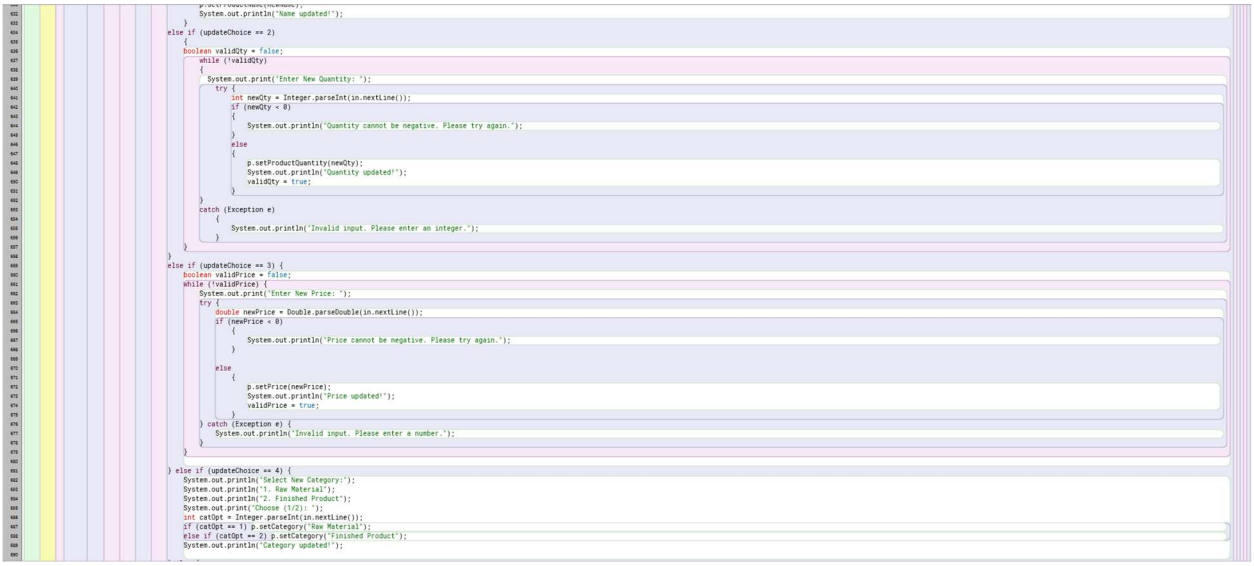
```
                System.out.printf("| %-10s | %-25s | %-8s | %-12s | %-20s |%n",
                        "ID", "Product Name", "Qty", "Price", "Category");
                System.out.println("-------------------------------------------------------------------------------------------------");

                while (!inventoryStack.isEmpty()) {
                    Product p = inventoryStack.pop();
                    boolean isMatch = false;

                    if (subChoice == 1) { // ID
                        if (p.getProductID().equalsIgnoreCase(keyword)) isMatch = true;
                    } else if (subChoice == 2) { // Name
                        if (p.getProductName().toLowerCase().contains(keyword.toLowerCase())) isMatch = true;
                    } else if (subChoice == 3) { // Quantity
                        if (String.valueOf(p.getProductQuantity()).equals(keyword)) isMatch = true;
                    } else if (subChoice == 4) { // Price (NEW)
                        try {
                            double targetPrice = Double.parseDouble(keyword);
                            if (p.getPrice() == targetPrice) isMatch = true;
                        } catch (NumberFormatException e) {
                            // Ignore invalid number format, just won't match
                        }
                    } else if (subChoice == 5) { // Category
                        if (p.getCategory().equalsIgnoreCase(keyword)) isMatch = true;
                    }

                    if (isMatch) {
                        System.out.printf("| %-10s | %-25s | %-8d | RM %-9.2f | %-20s |%n",
                                p.getProductID(), p.getProductName(),
                                p.getProductQuantity(), p.getPrice(), p.getCategory());
                        found = true;
                    }
                    tempStack.push(p);
                }
                System.out.println("-------------------------------------------------------------------------------------------------");

                while (!tempStack.isEmpty()) {
                    inventoryStack.push(tempStack.pop());
                }

                if (!found) {
                    System.out.println("No result found.");

                    boolean validResponse = false;
                    while (!validResponse) {
                        System.out.print("Do you want to search again? (Y/N): ");
                        String retry = in.nextLine();
                        if (retry.equalsIgnoreCase("Y")) {
                            validResponse = true;
                        } else if (retry.equalsIgnoreCase("N")) {
                            validResponse = true;
                            searchLoop = false;
                            System.out.println("Returning to Main Menu...");
                        } else {
                            System.out.println("Invalid input. Please enter 'Y' or 'N'.");
                        }
                    }
                } else {
                    boolean validResponse = false;
                    while (!validResponse) {
                        System.out.print("Search another item? (Y/N): ");
```



```
                        while (!validResponse) {
                            System.out.print("Search another item? (Y/N): ");
                            String retry = in.nextLine();
                            if (retry.equalsIgnoreCase("Y")) {
                                validResponse = true;
                            } else if (retry.equalsIgnoreCase("N")) {
                                validResponse = true;
                                searchLoop = false;
                                System.out.println("Returning to Main Menu...");
                            } else {
                                System.out.println("Invalid input. Please enter 'Y' or 'N'.");
                            }
                        }
                    }
                } else {
                    System.out.println("Invalid search option.");
                }
            }
        }
    }

    // ==========================================================
    // OPTION 5: UPDATE PRODUCT
    // ==========================================================
    else if (choice == 5) {

        if (inventoryStack.isEmpty()) {
            System.out.println("Empty, nothing to update.");
            System.out.println("\n[Press Enter to return to Main Menu...]");
            in.nextLine();
        } else {
            // PART 1: DISPLAY FULL TABLE
            System.out.println("\n--- LIST OF PRODUCTS AVAILABLE FOR UPDATE ---");
            System.out.println("-------------------------------------------------------------------------------------------------");
            System.out.printf("| %-10s | %-25s | %-8s | %-12s | %-20s |%n",
                    "ID", "Product Name", "Qty", "Price", "Category");
            System.out.println("-------------------------------------------------------------------------------------------------");

            while (!inventoryStack.isEmpty()) {
                Product p = inventoryStack.pop();
                System.out.printf("| %-10s | %-25s | %-8d | RM %-9.2f | %-20s |%n",
                                  p.getProductID(),
                                  p.getProductName(),
                                  p.getProductQuantity(),
                                  p.getPrice(),
                                  p.getCategory());
                tempStack.push(p);
            }
            while (!tempStack.isEmpty()) {
                inventoryStack.push(tempStack.pop());
            }
            System.out.println("-------------------------------------------------------------------------------------------------");

            // PART 2: UPDATE LOGIC
            boolean searchAgain = true;
            while (searchAgain) {
                System.out.print("Please enter Product ID to update (or type 'EXIT' to return): ");
                String targetId = in.nextLine();

                if (targetId.equalsIgnoreCase("EXIT"))
```



```
        String targetId = in.nextLine();

        if (targetId.equalsIgnoreCase("EXIT"))
        {
            searchAgain = false;
            System.out.println("Returning to Main Menu...");
            break;
        }

        boolean found = false;

        while (!inventoryStack.isEmpty()) {
            Product p = inventoryStack.pop();

            if (p.getProductID().equalsIgnoreCase(targetId)) {
                found = true;
                boolean updatingThisProduct = true;

                while (updatingThisProduct) {

                    System.out.println("\n--- SELECTED PRODUCT DETAILS ---");
                    System.out.println("-----------------------------------------------------------------------------------------");
                    System.out.printf("| %-10s | %-25s | %-8d | RM %-9.2f | %-20s |%n",
                            p.getProductID(),
                            p.getProductName(),
                            p.getProductQuantity(),
                            p.getPrice(),
                            p.getCategory());
                    System.out.println("-----------------------------------------------------------------------------------------");

                    System.out.println("WHAT DO YOU WANT TO UPDATE?");
                    System.out.println("1. Update Name");
                    System.out.println("2. Update Quantity");
                    System.out.println("3. Update Price");
                    System.out.println("4. Update Category");
                    System.out.println("0. Back to Main Menu");
                    System.out.print("Enter your choice (0-4): ");

                    int updateChoice = -1;
                    try {
                        updateChoice = Integer.parseInt(in.nextLine());
                    } catch (Exception e) {
                        updateChoice = -1;
                    }

                    if (updateChoice == 0) {
                        recentActivityStack.push(p);
                        updatingThisProduct = false;
                        searchAgain = false;
                        System.out.println("Returning to Main Menu...");
                        break;
                    }

                    else if (updateChoice == 1)
                        {
                            System.out.print("Enter New Name: ");
                            String newName = in.nextLine();
                            p.setProductName(newName);
                            System.out.println("Name updated!");
                        }
```



```
                            System.out.println("Name updated!");
                        }
                    else if (updateChoice == 2)
                        {
                        boolean validQty = false;
                            while (!validQty)
                            {
                             System.out.print("Enter New Quantity: ");
                                try {
                                    int newQty = Integer.parseInt(in.nextLine());
                                    if (newQty < 0)
                                    {
                                        System.out.println("Quantity cannot be negative. Please try again.");
                                    }
                                    else
                                    {
                                        p.setProductQuantity(newQty);
                                        System.out.println("Quantity updated!");
                                        validQty = true;
                                    }
                                }
                                catch (Exception e)
                                    {
                                        System.out.println("Invalid input. Please enter an integer.");
                                    }
                            }
                        }
                    else if (updateChoice == 3) {
                        boolean validPrice = false;
                        while (!validPrice) {
                            System.out.print("Enter New Price: ");
                            try {
                                double newPrice = Double.parseDouble(in.nextLine());
                                if (newPrice < 0)
                                    {
                                        System.out.println("Price cannot be negative. Please try again.");
                                    }

                                else
                                    {
                                        p.setPrice(newPrice);
                                        System.out.println("Price updated!");
                                        validPrice = true;
                                    }
                            } catch (Exception e) {
                                System.out.println("Invalid input. Please enter a number.");
                            }
                        }

                    } else if (updateChoice == 4) {
                        System.out.println("Select New Category:");
                        System.out.println("1. Raw Material");
                        System.out.println("2. Finished Product");
                        System.out.print("Choose (1/2): ");
                        int catOpt = Integer.parseInt(in.nextLine());
                        if (catOpt == 1) p.setCategory("Raw Material");
                        else if (catOpt == 2) p.setCategory("Finished Product");
                        System.out.println("Category updated!");
```

```
                    } else {
                        System.out.println("Invalid choice.");
                    }

                    if (updateChoice != 0) {
                        boolean validResponse = false;
                        while (!validResponse) {
                            System.out.print("\nDo you want to continue updating this product? (Y = Yes / N = Back to Menu): ");
                            String cont = in.nextLine();

                            if (cont.equalsIgnoreCase("Y")) {
                                validResponse = true;
                            } else if (cont.equalsIgnoreCase("N"))
                                {
                                    validResponse = true;
                                    updatingThisProduct = false;
                                    recentActivityStack.push(p);
                                    searchAgain = false;
                                    System.out.println("Returning to Main Menu...");
                                }
                            else
                                {
                                    System.out.println("Invalid input. Please enter 'Y' or 'N'.");
                                }
                        }
                    }
                }
                tempStack.push(p);
            }

            while (!tempStack.isEmpty()) {
                inventoryStack.push(tempStack.pop());
            }

            if (found)
                {
                    saveFile(inventoryStack);
                }
            else
                {
                    System.out.println("Product ID not found.");

                    boolean validResponse = false;
                    while (!validResponse) {
                        System.out.print("Try another ID? (Y = Yes / N = Back to Menu): ");
                        String answer = in.nextLine();
                        if (answer.equalsIgnoreCase("Y"))
                        {
                            validResponse = true;
                        }
                        else if (answer.equalsIgnoreCase("N"))
                        {
                            validResponse = true;
                            searchAgain = false;
                            System.out.println("Returning to Main Menu...");
                        }
```

```
                        System.out.println("Returning to Main Menu...");
                        }

                        else
                        {
                            System.out.println("Invalid input. Please enter 'Y' or 'N'.");
                        }
                    }
                }
            }
        }

        // ==========================================================
        // OPTION 6: VIEW TRANSACTION
        // ==========================================================
        else if (choice == 6) {
            if (recentActivityStack.isEmpty())
                {
                    System.out.println("No recent activity recorded.");
                }
            else
                {
                    System.out.println("\n--- --- INVENTORY UPDATES LOG --- ---");
                    while (!recentActivityStack.isEmpty())
                    {
                        Product p = recentActivityStack.pop();
                        System.out.println("Product: " + p.getProductName() + " (Last Edited)");
                        tempStack.push(p);
                    }
                    while (!tempStack.isEmpty())
                    {
                        recentActivityStack.push(tempStack.pop());
                    }
                }
            System.out.println("\n[Press Enter to return to Main Menu...]");
            in.nextLine();
        }

        else if (choice == 0)
            {
                System.out.println("Bye bye! Thank you.");
            }
        else
            {
                System.out.println("Invalid number selected.");
            }
    }
}
```

2.2 PRODUCT CLASS

Product 2026-Jan-23 20:18 Page 1

```java
public class Product {

    // --- Attributes
    private String productID;
    private String productName;
    private int productQuantity;
    private double price;
    private String category;

    // --- Constructor ---
    public Product(String productID, String productName, int productQuantity, double price, String category) {
        this.productID = productID;
        this.productName = productName;
        this.productQuantity = productQuantity;
        this.price = price;
        this.category = category;
    }

    // --- Setters (Mutators) ---
    public void setProductID(String productID) {
        this.productID = productID;
    }

    public void setProductName(String productName) {
        this.productName = productName;
    }

    public void setProductQuantity(int productQuantity) {
        this.productQuantity = productQuantity;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public void setCategory(String category) {
        this.category = category;
    }

    // --- Getters (Accessors) ---
    public String getProductID() {
        return productID;
    }

    public String getProductName() {
        return productName;
    }

    public int getProductQuantity() {
        return productQuantity;
    }

    public double getPrice() {
        return price;
    }

    public String getCategory() {
        return category;
    }

    // Returns a string representation of the object
    @Override
    public String toString() {
        return "Product [" +
                "ID='" + productID + '\'' +
                ", Name='" + productName + '\'' +
                ", Qty=" + productQuantity +
                ", Price=" + price +
                ", Category='" + category + '\'' +
                ']';
    }
}
```

2.3 LINKEDLIST CLASS

LinkedList 2026-Jan-23 20:40 Page 1

```
1  /**
2  Coder: Roslan S, UiTM Pahang, roslancs@uitm.edu.my
3  Year: 2012
4  Improved & Fixed Version
5  */
6
7  public class LinkedList<E> {
8
9      private Node<E> head, current, tail;
10
11     public LinkedList() {
12         head = current = tail = null;
13     }
14
15     public boolean isEmpty() {
16         return head == null;
17     }
18
19     // ================ ADD ================
20
21     public void addFirst(E data) {
22         Node<E> newNode = new Node<>(data); // Node dipanggil di sini
23         newNode.next = head;
24         head = newNode;
25
26         if (tail == null) {
27             tail = head;
28         }
29     }
30
31     public void addLast(E data) {
32         Node<E> newNode = new Node<>(data);
33
34         if (tail == null) {
35             head = tail = newNode;
36         } else {
37             tail.next = newNode;
38             tail = newNode;
39         }
40     }
41
42     public void add(int index, E data) {
43         if (index < 0)
44             throw new IndexOutOfBoundsException();
45
46         Node<E> newNode = new Node<>(data);
47
48         if (index == 0) {
49             newNode.next = head;
50             head = newNode;
51             if (tail == null)
52                 tail = head;
53             return;
54         }
55
56         Node<E> temp = head;
57         int count = 0;
58
59         while (temp != null && count < index - 1) {
60             temp = temp.next;
61             count++;
62         }
63
64         if (temp == null || temp.next == null) {
65             if (tail == null) {
66                 head = tail = newNode;
67             } else {
68                 tail.next = newNode;
69                 tail = newNode;
70             }
71         } else {
72             newNode.next = temp.next;
73             temp.next = newNode;
```

```
        }
    }

    // ================ GET ================

    public E getFirst() {
        if (isEmpty()) return null;
        current = head;
        return current.data;
    }

    public E getLast() {
        if (isEmpty()) return null;
        return tail.data;
    }

    public E getNext() {
        if (current == null || current == tail) return null;
        current = current.next;
        return current.data;
    }

    // ================ REMOVE ================

    public E removeFirst() {
        if (isEmpty()) return null;
        current = head;
        head = head.next;
        if (head == null) tail = null;
        return current.data;
    }

    public E removeLast() {
        if (isEmpty()) return null;
        if (head == tail) {
            current = head;
            head = tail = null;
            return current.data;
        }
        Node<E> temp = head;
        while (temp.next != tail) {
            temp = temp.next;
        }
        Node<E> removed = tail;
        tail = temp;
        tail.next = null;
        return removed.data;
    }

    public E removeAfter(E data) {
        if (isEmpty()) return null;
        Node<E> temp = head;
        while (temp != null && !data.equals(temp.data)) {
            temp = temp.next;
        }
        if (temp == null || temp.next == null) return null;
        Node<E> removed = temp.next;
        temp.next = removed.next;
        if (removed == tail) tail = temp;
        return removed.data;
    }

    // ================ SEARCH & UTILITY ================

    public boolean contains(E data) {
        Node<E> temp = head;
        while (temp != null) {
            if (data.equals(temp.data)) return true;
            temp = temp.next;
        }
        return false;
    }

```

```
147     public void clear() {
148         head = current = tail = null;
149     }
150 
151     @Override
152     public String toString() {
153         StringBuilder result = new StringBuilder("[");
154         if (isEmpty()) {
155             result.append("The list is empty]");
156             return result.toString();
157         }
158         Node<E> temp = head;
159         while (temp != null) {
160             result.append(temp.data);
161             temp = temp.next;
162             if (temp != null) result.append(", ");
163             else result.append("]");
164         }
165         return result.toString();
166     }
167 
168     // ================ CLASS NODE (PENYELESAIAN MASALAH) ================
169     // Letak class ini di dalam LinkedList supaya tak error
170     private static class Node<E> {
171         E data;
172         Node<E> next;
173 
174         public Node(E data) {
175             this.data = data;
176             this.next = null;
177         }
178     }
179 }
```

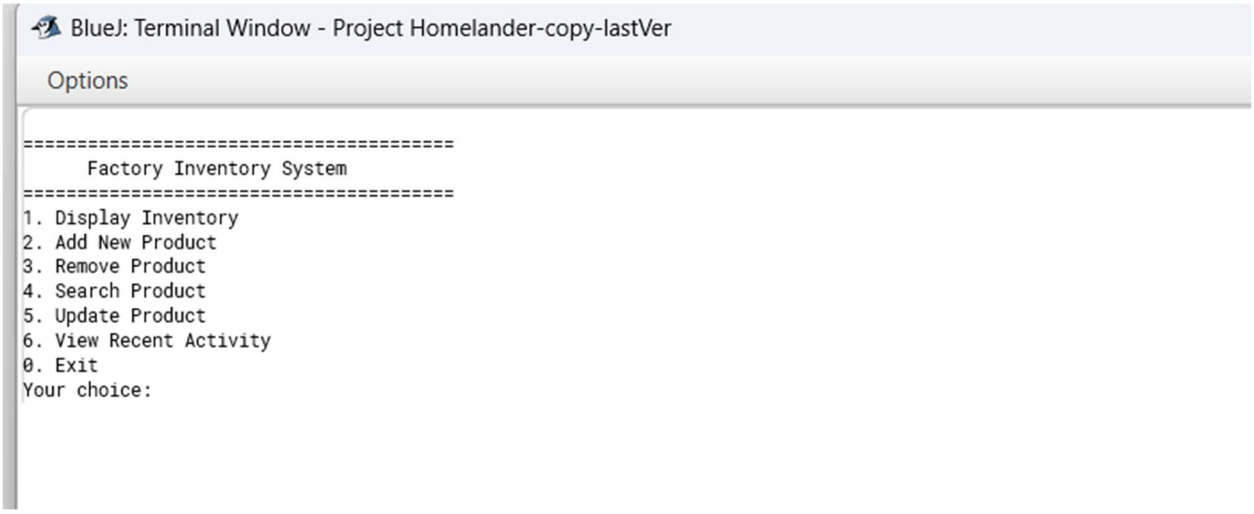
2.4 STACK CLASS

Stack 2026-Jan-23 20:42 Page 1

```java
import java.util.ArrayList;

public class Stack<E> {
    private ArrayList<E> stack;

    // create an empty stack
    public Stack() {
        stack = new ArrayList<>();
    }

    // return true if stack has no elements
    public boolean isEmpty() {
        return stack.isEmpty();
    }

    // insert item onto the top of the stack
    public E push(E item) {
        stack.add(item);
        return item;
    }

    // remove and return the top item
    public E pop() {
        if (isEmpty()) {
            throw new RuntimeException("Stack is empty");
        }
        return stack.remove(stack.size() - 1);
    }

    // return the top item without removing it
    public E peek() {
        if (isEmpty()) {
            throw new RuntimeException("Stack is empty");
        }
        return stack.get(stack.size() - 1);
    }
}
```

3.0 EXPECTED OUTPUT

3.1 Menu



3.2 Display Data (Process 1)

```
Your choice: 1

--- PRODUCT LIST (STACK) ---
-------------------------------------------------------------------------------------------
| ID         | Product Name              | Qty      | Price        | Category             |
-------------------------------------------------------------------------------------------
| 321        | grab                      | 23       | RM 32.00     | Raw Material         |
| 142        | bomb                      | 22       | RM 34.00     | Finished Product     |
| 212        | wood track                | 6        | RM 23.00     | Raw Material         |
| 233        | wood plank                | 32       | RM 2.00      | Raw Material         |
| 130        | Wooden Rack               | 30       | RM 45.00     | Finished Product     |
| 124        | ikan talapia              | 34       | RM 67.00     | Finished Product     |
| 132        | Nasi Lemak PAttaya        | 23       | RM 67.00     | Finished Product     |
| 143        | Ikan Masak Keke           | 23       | RM 67.00     | Finished Product     |
| 101        | Aluminium Sheet           | 45       | RM 30.00     | Finished Product     |
| 103        | Copper Wire               | 5000     | RM 5.50      | Raw Material         |
| 104        | Plastic Resin             | 75       | RM 3.00      | Raw Material         |
| 106        | Glass Panel               | 60       | RM 20.00     | Raw Material         |
| 107        | Lubricant Oil             | 80       | RM 10.00     | Raw Material         |
| 108        | Iron Nail                 | 500      | RM 0.20      | Raw Material         |
| 109        | Cement Bag 50kg           | 100      | RM 25.00     | Raw Material         |
| 110        | Wood Plank                | 70       | RM 15.00     | Raw Material         |
| 111        | LED Light Bulb            | 120      | RM 15.00     | Finished Product     |
| 112        | Electric Fan              | 80       | RM 45.00     | Finished Product     |
| 113        | Plastic Container         | 200      | RM 2.50      | Finished Product     |
| 114        | Steel Screw Set           | 500      | RM 0.50      | Finished Product     |
| 115        | Copper Pipe 1m            | 60       | RM 12.00     | Finished Product     |
| 116        | Wooden Chair              | 40       | RM 85.00     | Finished Product     |
| 117        | Aluminium Window Frame    | 30       | RM 120.00    | Finished Product     |
| 118        | Metal Shelf Rack          | 25       | RM 150.00    | Finished Product     |
| 119        | Plastic Water Tank 500L   | 40       | RM 35.00     | Finished Product     |
| 121        | minyak masak              | 23       | RM 43.00     | Finished Product     |
-------------------------------------------------------------------------------------------

[Press Enter to return to Main Menu...]
```

3.3 Add New Product (Process 2)

```
========================================
      Factory Inventory System
========================================
1. Display Inventory
2. Add New Product
3. Remove Product
4. Search Product
5. Update Product
6. View Recent Activity
0. Exit
Your choice: 2

Enter Product ID: 90
Product Name: Vibranium
Quantity: 1
Price: 240
Category: 1.Raw Material, 2.Finished Product
Choose (1/2):
1
Product added successfully!

Add another product? (Y/N): N
Returning to Main Menu...
```

3.4 After Adding New Product (Process 2)

```
--- PRODUCT LIST (STACK) ---
-------------------------------------------------------------------------------------------
| ID         | Product Name              | Qty      | Price        | Category             |
-------------------------------------------------------------------------------------------
| 90         | Vibranium                 | 1        | RM 240.00    | Raw Material         |
| 321        | grab                      | 23       | RM 32.00     | Raw Material         |
| 142        | bomb                      | 22       | RM 34.00     | Finished Product     |
| 212        | wood track                | 6        | RM 23.00     | Raw Material         |
| 233        | wood plank                | 32       | RM 2.00      | Raw Material         |
| 130        | Wooden Rack               | 30       | RM 45.00     | Finished Product     |
| 124        | ikan talapia              | 34       | RM 67.00     | Finished Product     |
| 132        | Nasi Lemak PAttaya        | 23       | RM 67.00     | Finished Product     |
| 143        | Ikan Masak Keke           | 23       | RM 67.00     | Finished Product     |
| 101        | Aluminium Sheet           | 45       | RM 30.00     | Finished Product     |
| 103        | Copper Wire               | 5000     | RM 5.50      | Raw Material         |
| 104        | Plastic Resin             | 75       | RM 3.00      | Raw Material         |
| 106        | Glass Panel               | 60       | RM 20.00     | Raw Material         |
| 107        | Lubricant Oil             | 80       | RM 10.00     | Raw Material         |
| 108        | Iron Nail                 | 500      | RM 0.20      | Raw Material         |
| 109        | Cement Bag 50kg           | 100      | RM 25.00     | Raw Material         |
| 110        | Wood Plank                | 70       | RM 15.00     | Raw Material         |
| 111        | LED Light Bulb            | 120      | RM 15.00     | Finished Product     |
| 112        | Electric Fan              | 80       | RM 45.00     | Finished Product     |
| 113        | Plastic Container         | 200      | RM 2.50      | Finished Product     |
| 114        | Steel Screw Set           | 500      | RM 0.50      | Finished Product     |
| 115        | Copper Pipe 1m            | 60       | RM 12.00     | Finished Product     |
| 116        | Wooden Chair              | 40       | RM 85.00     | Finished Product     |
| 117        | Aluminium Window Frame    | 30       | RM 120.00    | Finished Product     |
| 118        | Metal Shelf Rack          | 25       | RM 150.00    | Finished Product     |
| 119        | Plastic Water Tank 500L   | 40       | RM 35.00     | Finished Product     |
| 121        | minyak masak              | 23       | RM 43.00     | Finished Product     |
-------------------------------------------------------------------------------------------

[Press Enter to return to Main Menu...]
```

3.5 Remove (Process 3)

```
=======================================
      Factory Inventory System
=======================================
1. Display Inventory
2. Add New Product
3. Remove Product
4. Search Product
5. Update Product
6. View Recent Activity
0. Exit
Your choice: 3

--- LIST OF PRODUCTS AVAILABLE TO REMOVE ---
-------------------------------------------------------------------------------------------
| ID         | Product Name              | Qty      | Price        | Category             |
-------------------------------------------------------------------------------------------
| 90         | Vibranium                 | 1        | RM 240.00    | Raw Material         |
| 321        | grab                      | 23       | RM 32.00     | Raw Material         |
| 142        | bomb                      | 22       | RM 34.00     | Finished Product     |
| 212        | wood track                | 6        | RM 23.00     | Raw Material         |
| 233        | wood plank                | 32       | RM 2.00      | Raw Material         |
| 130        | Wooden Rack               | 30       | RM 45.00     | Finished Product     |
| 124        | ikan talapia              | 34       | RM 67.00     | Finished Product     |
| 132        | Nasi Lemak PAttaya        | 23       | RM 67.00     | Finished Product     |
| 143        | Ikan Masak Keke           | 23       | RM 67.00     | Finished Product     |
| 101        | Aluminium Sheet           | 45       | RM 30.00     | Finished Product     |
| 103        | Copper Wire               | 5000     | RM 5.50      | Raw Material         |
| 104        | Plastic Resin             | 75       | RM 3.00      | Raw Material         |
| 106        | Glass Panel               | 60       | RM 20.00     | Raw Material         |
| 107        | Lubricant Oil             | 80       | RM 10.00     | Raw Material         |
| 108        | Iron Nail                 | 500      | RM 0.20      | Raw Material         |
| 109        | Cement Bag 50kg           | 100      | RM 25.00     | Raw Material         |
| 110        | Wood Plank                | 70       | RM 15.00     | Raw Material         |
| 111        | LED Light Bulb            | 120      | RM 15.00     | Finished Product     |
| 112        | Electric Fan              | 80       | RM 45.00     | Finished Product     |
| 113        | Plastic Container         | 200      | RM 2.50      | Finished Product     |
| 114        | Steel Screw Set           | 500      | RM 0.50      | Finished Product     |
| 115        | Copper Pipe 1m            | 60       | RM 12.00     | Finished Product     |
| 116        | Wooden Chair              | 40       | RM 85.00     | Finished Product     |
| 117        | Aluminium Window Frame    | 30       | RM 120.00    | Finished Product     |
| 118        | Metal Shelf Rack          | 25       | RM 150.00    | Finished Product     |
```

3.6 Remove continuation (Process 3)

ID	Product Name	Qty	Price	Category
90	Vibranium	1	RM 240.00	Raw Material
321	grab	23	RM 32.00	Raw Material
142	bomb	22	RM 34.00	Finished Product
212	wood track	6	RM 23.00	Raw Material
233	wood plank	32	RM 2.00	Raw Material
130	Wooden Rack	30	RM 45.00	Finished Product
124	ikan talapia	34	RM 67.00	Finished Product
132	Nasi Lemak PAttaya	23	RM 67.00	Finished Product
143	Ikan Masak Keke	23	RM 67.00	Finished Product
101	Aluminium Sheet	45	RM 30.00	Finished Product
103	Copper Wire	5000	RM 5.50	Raw Material
104	Plastic Resin	75	RM 3.00	Raw Material
106	Glass Panel	60	RM 20.00	Raw Material
107	Lubricant Oil	80	RM 10.00	Raw Material
108	Iron Nail	500	RM 0.20	Raw Material
109	Cement Bag 50kg	100	RM 25.00	Raw Material
110	Wood Plank	70	RM 15.00	Raw Material
111	LED Light Bulb	120	RM 15.00	Finished Product
112	Electric Fan	80	RM 45.00	Finished Product
113	Plastic Container	200	RM 2.50	Finished Product
114	Steel Screw Set	500	RM 0.50	Finished Product
115	Copper Pipe 1m	60	RM 12.00	Finished Product
116	Wooden Chair	40	RM 85.00	Finished Product
117	Aluminium Window Frame	30	RM 120.00	Finished Product
118	Metal Shelf Rack	25	RM 150.00	Finished Product
119	Plastic Water Tank 500L	40	RM 35.00	Finished Product
121	minyak masak	23	RM 43.00	Finished Product

```
Enter Product ID to remove: 90

Product Found: Vibranium
Are you sure you want to delete? (Y/N): Y
Product removed from inventory.
Successfully updated database: Vibranium is gone.

Remove another product? (Y = Yes / N = Back to Menu): N
Returning to Main Menu...
```

3.7 Search Product (Process 4)

```
=======================================
      Factory Inventory System
=======================================
1. Display Inventory
2. Add New Product
3. Remove Product
4. Search Product
5. Update Product
6. View Recent Activity
0. Exit
Your choice: 4

--- SEARCH MENU ---
1. Search by ID
2. Search by Product Name
3. Search by Quantity
4. Search by Price
5. Search by Category
0. Back to Main Menu
Enter choice: 5

Select Category to search:
1. Raw Material
2. Finished Product
Enter choice (1/2): 1

--- SEARCH RESULTS (Raw Material) ---
-----------------------------------------------------------------------------------------------
| ID         | Product Name              | Qty      | Price        | Category             |
-----------------------------------------------------------------------------------------------
| 321        | grab                      | 23       | RM 32.00     | Raw Material         |
| 212        | wood track                | 6        | RM 23.00     | Raw Material         |
| 233        | wood plank                | 32       | RM 2.00      | Raw Material         |
| 103        | Copper Wire               | 5000     | RM 5.50      | Raw Material         |
| 104        | Plastic Resin             | 75       | RM 3.00      | Raw Material         |
| 106        | Glass Panel               | 60       | RM 20.00     | Raw Material         |
| 107        | Lubricant Oil             | 80       | RM 10.00     | Raw Material         |
| 108        | Iron Nail                 | 500      | RM 0.20      | Raw Material         |
| 109        | Cement Bag 50kg           | 100      | RM 25.00     | Raw Material         |
| 110        | Wood Plank                | 70       | RM 15.00     | Raw Material         |
-----------------------------------------------------------------------------------------------
Search another item? (Y/N):
```

3.8 Update Product (Process 5)

```
=======================================
      Factory Inventory System
=======================================
1. Display Inventory
2. Add New Product
3. Remove Product
4. Search Product
5. Update Product
6. View Recent Activity
0. Exit
Your choice: 5

--- LIST OF PRODUCTS AVAILABLE FOR UPDATE ---
-------------------------------------------------------------------------------------------
| ID         | Product Name               | Qty      | Price        | Category             |
-------------------------------------------------------------------------------------------
| 321        | grab                       | 23       | RM 32.00     | Raw Material         |
| 142        | bomb                       | 22       | RM 34.00     | Finished Product     |
| 212        | wood track                 | 6        | RM 23.00     | Raw Material         |
| 233        | wood plank                 | 32       | RM 2.00      | Raw Material         |
| 130        | Wooden Rack                | 30       | RM 45.00     | Finished Product     |
| 124        | ikan talapia               | 34       | RM 67.00     | Finished Product     |
| 132        | Nasi Lemak PAttaya         | 23       | RM 67.00     | Finished Product     |
| 143        | Ikan Masak Keke            | 23       | RM 67.00     | Finished Product     |
| 101        | Aluminium Sheet            | 45       | RM 30.00     | Finished Product     |
| 103        | Copper Wire                | 5000     | RM 5.50      | Raw Material         |
| 104        | Plastic Resin              | 75       | RM 3.00      | Raw Material         |
| 106        | Glass Panel                | 60       | RM 20.00     | Raw Material         |
| 107        | Lubricant Oil              | 80       | RM 10.00     | Raw Material         |
| 108        | Iron Nail                  | 500      | RM 0.20      | Raw Material         |
| 109        | Cement Bag 50kg            | 100      | RM 25.00     | Raw Material         |
| 110        | Wood Plank                 | 70       | RM 15.00     | Raw Material         |
| 111        | LED Light Bulb             | 120      | RM 15.00     | Finished Product     |
| 112        | Electric Fan               | 80       | RM 45.00     | Finished Product     |
| 113        | Plastic Container          | 200      | RM 2.50      | Finished Product     |
| 114        | Steel Screw Set            | 500      | RM 0.50      | Finished Product     |
| 115        | Copper Pipe 1m             | 60       | RM 12.00     | Finished Product     |
| 116        | Wooden Chair               | 40       | RM 85.00     | Finished Product     |
| 117        | Aluminium Window Frame     | 30       | RM 120.00    | Finished Product     |
| 118        | Metal Shelf Rack           | 25       | RM 150.00    | Finished Product     |
| 119        | Plastic Water Tank 500L    | 40       | RM 35.00     | Finished Product     |
| 121        | minyak masak               | 23       | RM 43.00     | Finished Product     |
```

3.9 Update Product Continuation (Process 5)

```
| 115        | Copper Pipe 1m             | 60       | RM 12.00     | Finished Product     |
| 116        | Wooden Chair               | 40       | RM 85.00     | Finished Product     |
| 117        | Aluminium Window Frame     | 30       | RM 120.00    | Finished Product     |
| 118        | Metal Shelf Rack           | 25       | RM 150.00    | Finished Product     |
| 119        | Plastic Water Tank 500L    | 40       | RM 35.00     | Finished Product     |
| 121        | minyak masak               | 23       | RM 43.00     | Finished Product     |
-------------------------------------------------------------------------------------------
Please enter Product ID to update (or type 'EXIT' to return): 321

--- SELECTED PRODUCT DETAILS ---
-------------------------------------------------------------------------------------------
| 321        | grab                       | 23       | RM 32.00     | Raw Material         |
-------------------------------------------------------------------------------------------
WHAT DO YOU WANT TO UPDATE?
1. Update Name
2. Update Quantity
3. Update Price
4. Update Category
0. Back to Main Menu
Enter your choice (0-4): 1
Enter New Name: Oak wood
Name updated!

Do you want to continue updating this product? (Y = Yes / N = Back to Menu): Y

--- SELECTED PRODUCT DETAILS ---
-------------------------------------------------------------------------------------------
| 321        | Oak wood                   | 23       | RM 32.00     | Raw Material         |
-------------------------------------------------------------------------------------------
WHAT DO YOU WANT TO UPDATE?
1. Update Name
2. Update Quantity
3. Update Price
4. Update Category
0. Back to Main Menu
Enter your choice (0-4): 2
Enter New Quantity: 30
Quantity updated!

Do you want to continue updating this product? (Y = Yes / N = Back to Menu): n
Returning to Main Menu...
```

3.10 View Recent Activity (Process 6)

```
========================================
      Factory Inventory System
========================================
1. Display Inventory
2. Add New Product
3. Remove Product
4. Search Product
5. Update Product
6. View Recent Activity
0. Exit
Your choice: 6

--- --- INVENTORY UPDATES LOG --- ---
Product: Oak wood (Last Edited)
Product: Vibranium (Last Edited)
Product: Vibranium (Last Edited)

[Press Enter to return to Main Menu...]
```

3.11 Exit

```
========================================
      Factory Inventory System
========================================
1. Display Inventory
2. Add New Product
3. Remove Product
4. Search Product
5. Update Product
6. View Recent Activity
0. Exit
Your choice: 0
Bye bye! Thank you.
```

4.0 CONCLUSION

In conclusion, the Factory Inventory Management System was developed to address common challenges faced by factories in managing inventory, such as inaccurate records, inefficient tracking, and delays in updating stock information. By providing a centralized platform for storing and managing product data, the system helps improve organization, visibility, and control over inventory records. This reduces reliance on manual methods and scattered files, which often lead to errors and inefficiencies in daily operations.

The system supports essential inventory tasks, including adding, updating, removing, and searching for products, allowing users to manage inventory information more efficiently. With structured data handling and consistent record updates, the system contributes to better decision-making and smoother factory operations. The integration of basic data management principles also ensures that inventory information remains accurate and up to date.

Overall, this project demonstrates how a simple digital solution can enhance factory inventory management and support more efficient industrial practices. By promoting better data organization and resource monitoring, the system aligns with the objectives of Sustainable Development Goal 9, which emphasizes industry innovation and improved infrastructure. Although the system is basic in scope, it provides a strong foundation for future improvements and serves as a practical step toward more effective and sustainable inventory management.

5.0 REFERENCES

1. **United Nations. (n.d.).** ***United Nations Sustainable Development Goals.*** **https://sdgs.un.org/goals**
2. **w3schools. (n.d.).** ***Linked list.*** **https://www.w3schools.com/dsa/dsa_theory_linkedlists.php**
3. **w3schools. (n.d.).** ***Stack.*** **https://www.w3schools.com/dsa/dsa_data_stacks.php**
4. **w3schools. (n.d.).** ***Java file handling.*** **https://www.w3schools.com/java/java_files.asp**
5. **IBM. (n.d.).** ***What is inventory management?*** **https://www.ibm.com/topics/inventory-management**